

Main chapter ko concept-by-concept explain karunga: pehle mental model, phir four binding rules, arrow functions, bind, callbacks, aur tricky edge cases. Saath mein har major section ke baad practical rule aur small examples dunga. # Chapter 4: "This Works" ka detailed Hinglish explanation

Is chapter ka central idea bahut simple hai, lekin uske consequences thode complex hain:

Regular JavaScript function ke andar this kya hoga, ye function likhte waqt decide nahi hota. Ye function ko call karne ke tareeke se runtime par decide hota hai.

Isliye this ko samajhne ke liye function definition se zyada important uska **call-site**, yani function invocation, hota hai.

1. Sabse pehle this ka correct mental model

Bahut log sochte hain:

- this current function ko refer karta hai
- this us object ko refer karta hai jisme function likha gaya hai
- class ka method hai, to this automatically class instance hi hoga
- function jahan define hua hai, this us surrounding object ko point karega

Regular functions ke case mein ye assumptions reliable nahi hain.

Consider:

```
1  const user = {  
2      name: "Ruth",  
3  
4      introduce() {  
5          console.log(this.name);  
6      }  
7  };
```

Sirf function definition dekhkar hum ye nahi keh sakte ki this hamesha user hoga.

Reason:

```
1  user.introduce();
```

Yahan this === user hoga.

Lekin:

```
1  const fn = user.introduce;  
2  fn();
```

Ab function wahi hai, lekin call karne ka tareeka badal gaya. Isliye this bhi badal gaya.

Important distinction

Ye question incomplete hai:

introduce() ke andar this kya hai?

Correct question hai:

Jab introduce() ko **is particular tareeke se invoke kiya gaya**, tab us invocation mein this kya assign hoga?

Ek hi function alag-alag calls mein alag this receive kar sakta hai.

2. this-aware function kya hota hai?

Chapter ek useful terminology introduce karta hai:

Jis function ke body mein this use hua hai, woh this-aware function hai.

Example:

```
1 function greet() {  
2     console.log("Hello");  
3 }
```

Ye this-aware nahi hai, kyunki ismein this use nahi hua.

```
1 function greet() {  
2     console.log(`Hello ${this.name}`);  
3 }
```

Ye this-aware hai.

Pehle function ko call karte waqt this kya hai, isse koi farq nahi padta.

Doosre function ka behavior samajhne ke liye humein ye jaanana hi padega ki call ke time this kya tha.

3. this ko implicit parameter ki tarah samjho

Suppose:

```
1 function introduce(message) {  
2     console.log(message, this.name);  
3 }
```

Is function ka explicit parameter hai:

```
1 message
```

Lekin this ko ek hidden ya implicit parameter ki tarah samjha ja sakta hai.

Conceptually:

```
1 introduce(message, hiddenThis)
```

Actual JavaScript syntax aisa nahi hota, lekin mental model ke liye useful hai.

Difference ye hai ki explicit parameter clearly dikh raha hai:

```
1 function introduce(message)
```

Lekin this function header mein nahi dikhta. Pura function body padhne ke baad pata chalega ki function this use karta hai.

Isi wajah se this code ko understand karna kabhi-kabhi parameters se zyada difficult hota hai.

4. Lexical scope aur dynamic this context

Chapter yahan ek important comparison karta hai.

Lexical scope

Lexical scope code likhne ki location se decide hota hai.

```
1 const name = "Outer";  
2  
3 function showName() {  
4     console.log(name);  
5 }
```

showName() mein name ka resolution is baat par depend karega ki function code mein **kahan define hua tha**.

Function ko kahin aur call karne se lexical scope nahi badlega.

```
1 const name = "Outer";  
2  
3 function showName() {  
4     console.log(name);  
5 }  
6  
7 const obj = {  
8     name: "Object",  
9     showName
```

```
10  };
11
12  obj.showName(); // Outer
```

name ek lexical variable hai. obj.showName() call karne se lexical variable name object ki property nahi ban gaya.

Dynamic this context

Regular function ka this function ki definition location se decide nahi hota. Woh invocation se decide hota hai.

```
1  function showName() {
2      console.log(this.name);
3  }
4
5  const one = {
6      name: "One",
7      showName
8  };
9
10 const two = {
11     name: "Two",
12     showName
13 };
14
15 one.showName(); // One
16 two.showName(); // Two
```

Same function hai, lekin invocation context alag hai.

Isliye:

- lexical variables ka context mostly static hota hai
- regular function ka this context dynamic hota hai

Chapter ka point hai ki dynamic this powerful hai, lekin is flexibility ki wajah se complexity bhi aati hai.

5. point example

Chapter mein ye object diya gaya hai:

```
1  var point = {
```

```

2    x: null,
3    y: null,
4
5    init(x, y) {
6        this.x = x;
7        this.y = y;
8    },
9
10   rotate(angleRadians) {
11       var rotatedX =
12           this.x * Math.cos(angleRadians) -
13           this.y * Math.sin(angleRadians);
14
15       var rotatedY =
16           this.x * Math.sin(angleRadians) +
17           this.y * Math.cos(angleRadians);
18
19       this.x = rotatedX;
20       this.y = rotatedY;
21   },
22
23   toString() {
24       return `${this.x},${this.y}`;
25   },
26 };

```

Teen functions this-aware hain:

```

1  init()
2  rotate()
3  toString()

```

Lekin iska matlab ye nahi ki inmein this automatically point hoga.

Ye depend karega ki functions ko call kaise kiya gaya.

6. Rule 1: Implicit context invocation

```

1  point.init(3, 4);

```

Yahan call-site par function se pehle object reference hai:

```

1  point.

```

Is pattern mein dot ke left side wala object this banta hai.

Conceptually:

```
1 point.init(3, 4);
```

ke andar:

```
1 this === point
```

Isliye:

```
1 this.x = 3;  
2 this.y = 4;
```

effectively:

```
1 point.x = 3;  
2 point.y = 4;
```

ban jata hai.

Ek aur example

```
1 const person = {  
2   name: "Ruth",  
3  
4   showName() {  
5     console.log(this.name);  
6   }  
7 };  
8  
9 person.showName();
```

Result:

```
1 Ruth
```

Call-site:

```
1 person.showName()
```

Isliye this person hai.

Important subtlety

Function kiske andar originally define hua tha, woh important nahi hai. Call-site par dot ke left side ka object important hai.

```
1 const person1 = {  
2   name: "Person 1",  
3  
4   showName() {
```

```
5     console.log(this.name);
6   }
7 };
8
9 const person2 = {
10   name: "Person 2",
11   showName: person1.showName
12 };
13
14 person2.showName();
```

Output:

```
1 Person 2
```

Function person1 mein define hua tha, lekin call hua:

```
1 person2.showName()
```

Isliye this === person2.

7. Rule 2: Default context invocation

Ab method ka reference alag variable mein store karte hain:

```
1 const init = point.init;
2 init(3, 4);
```

Yahan function call hai:

```
1 init(3, 4)
```

Call-site par:

- koi object prefix nahi hai
- call() ya apply() nahi hai
- new nahi hai

Isliye default context rule lagega.

Strict mode mein

Modern JavaScript mein strict mode common hai. ES Modules aur class bodies strict mode mein run karte hain.

Strict mode mein plain function call ka this hota hai:

```
1 undefined
```

Example:

```
1 "use strict";
2
3 const init = point.init;
4 init(3, 4);
```

init() ke andar:

```
1 this.x = x;
```

effectively banega:

```
1 undefined.x = 3;
```

Isliye error:

```
1 TypeError: Cannot set properties of undefined
```

Method lose kaise hua?

Ye line function ko call nahi kar rahi:

```
1 const init = point.init;
```

Ye sirf function reference copy kar rahi hai.

Original association preserve nahi hoti:

```
1 point.init
```

se function nikalkar:

```
1 init
```

mein rakh diya.

Baad mein jab call hua:

```
1 init()
```

to call-site par point. present nahi hai.

Is phenomenon ko commonly **losing** this ya **lost context** kaha ja sakta hai.

8. Non-strict mode ka dangerous behavior

Non-strict mode mein plain function invocation ke liye default this global object ho sakta hai:

```
1 globalThis
```

Browser environment mein global object commonly window hai. Chapter ke context mein:

```
1 globalThis.x
```

set ho jayega.

Example:

```
1  const init = point.init;
2  init(3, 4);
```

Agar invocation non-strict behavior follow kare, to:

```
1  globalThis.x = 3;
2  globalThis.y = 4;
```

jabki:

```
1  point.x
2  point.y
```

unchanged reh sakte hain.

Ye dangerous hai kyunki:

1. expected object update nahi hua
2. global object pollute ho gaya
3. error immediately nahi aaya
4. bug program mein baad mein strange behavior dikha sakta hai

Isliye chapter strict mode ko prefer karne ko kehta hai. Strict mode mistake ko hidden global mutation banane ke bajay visible error bana deta hai.

9. Rule 3: Explicit context with call() and apply()

JavaScript functions ke paas call() aur apply() methods hote hain.

```
1  init.call(point, 3, 4);
```

call() ka first argument this value hota hai.

Yahan:

```
1  this === point
```

Baaki arguments original function ko milte hain:

```
1  x = 3
2  y = 4
```

Equivalent behavior:

```
1  point.init(3, 4);
```

call() syntax

```
1  functionName.call(thisValue, arg1, arg2, arg3);
```

Example:

```
1 function introduce(greeting, punctuation) {
2     console.log(`${greeting}, ${this.name}${punctuation}`);
3 }
4
5 const user = {
6     name: "Ruth"
7 };
8
9 introduce.call(user, "Hello", "!");
```

Output:

```
1 Hello, Ruth!
```

apply() syntax

apply() bhi first argument mein this leta hai, lekin function arguments array ya array-like collection mein leta hai:

```
1 functionName.apply(thisValue, [arg1, arg2, arg3]);
```

Example:

```
1 introduce.apply(user, ["Hello", "!"]);
```

Main difference:

```
1 fn.call(obj, a, b, c);
2 fn.apply(obj, [a, b, c]);
```

10. Method borrowing

Explicit context ki wajah se ek object ka function doosre object par use kar sakte hain.

```
1 const point = {
2     init(x, y) {
3         this.x = x;
4         this.y = y;
5     }
6 };
7
8 const anotherPoint = {};
9
10 point.init.call(anotherPoint, 5, 6);
```

init() originally point par hai, lekin invocation ke time explicitly kaha gaya:

```
1  this = anotherPoint
```

Isliye:

```
1  anotherPoint.x; // 5
2  anotherPoint.y; // 6
```

Is concept ko method borrowing samajh sakte hain.

Function physically copy nahi hua. Existing function ko different this ke saath call kiya gaya.

Borrowing through implicit context

Doosra option:

```
1  const anotherPoint = {
2      init: point.init
3  };
4
5  anotherPoint.init(5, 6);
```

Ab call-site hai:

```
1  anotherPoint.init()
```

Isliye implicit rule ke through this === anotherPoint.

Difference:

Explicit borrowing

```
1  point.init.call(anotherPoint, 5, 6);
```

Target object ko modify karna necessary nahi hai.

Property copy karke borrowing

```
1  anotherPoint.init = point.init;
2  anotherPoint.init(5, 6);
```

Target object par property add karni padti hai.

11. Rule 4: new context invocation

Regular constructable function ko new ke saath invoke kar sakte hain:

```
1  function Point(x, y) {
2      this.x = x;
3      this.y = y;
```

```
4 }  
5  
6 const point = new Point(3, 4);
```

new sirf class ke liye nahi hai. Ye functions ke saath bhi JavaScript ka underlying mechanism hai.

new ke four steps

Jab ye execute hota hai:

```
1 const point = new Point(3, 4);
```

JavaScript approximately chaar kaam karta hai.

Step 1: Naya empty object create karta hai

Conceptually:

```
1 const temp = {};
```

Step 2: Naye object ka prototype link set karta hai

```
1 Object.setPrototypeOf(temp, Point.prototype);
```

Conceptually:

```
1 temp → Point.prototype
```

Step 3: Function ko naye object ke this ke saath invoke karta hai

```
1 Point.call(temp, 3, 4);
```

Function ke andar:

```
1 this.x = 3;  
2 this.y = 4;
```

actually naye object ki properties set karega.

Step 4: Naya object return karta hai

Agar function explicitly apna object return nahi karta, to JavaScript created object return kar deta hai.

```
1 const point = temp;
```

12. new ke saath explicit return ka issue

Consider:

```
1 function Point(x, y) {  
2   this.x = x;
```

```
3     this.y = y;
4
5     return {
6         message: "Different object"
7     };
8 }
9
10 const point = new Point(3, 4);
```

Function ne khud ek object return kar diya. Is situation mein new se create hua object discard ho sakta hai aur explicitly returned object result ban jata hai.

Result conceptually:

```
1 point = {
2     message: "Different object"
3 };
```

Expected x aur y result mein nahi milenge.

Isi liye chapter advice deta hai ki new ke saath use hone wale functions mein explicit object return karna confusing ho sakta hai.

Important technical nuance

Chapter ka simplified manual version check karta hai:

```
1 typeof res !== "object" ? tmpObj : res
```

Real behavior ko samajhne ke liye itna yaad rakho:

- explicitly returned object ya function result replace kar sakta hai
- primitive return generally created object ko replace nahi karta

Core lesson explicit object return wala hai.

13. Concise methods aur new

Object literal mein concise method:

```
1 const point = {
2     init(x, y) {
3         this.x = x;
4         this.y = y;
5     }
6 };
```

Is form ko new ke saath call nahi kiya ja sakta:

```
1 new point.init(3, 4);
```

Ye error de sakta hai because concise methods constructable functions nahi hote.

Constructable function expression:

```
1 const point = {  
2   init: function (x, y) {  
3     this.x = x;  
4     this.y = y;  
5   }  
6 };
```

Ab:

```
1 const anotherPoint = new point.init(3, 4);
```

valid ho sakta hai.

Isliye har function-like syntax ko new ke saath use nahi kiya ja sakta.

14. Four this rules aur precedence

Regular functions ke liye chapter rules ko is precedence order mein rakhta hai:

1. new binding

```
1 new SomeFunction();
```

Naya object this banta hai.

2. Explicit binding

```
1 fn.call(obj);  
2 fn.apply(obj, []);
```

Provided object this banta hai.

3. Implicit binding

```
1 obj.fn();
```

Dot ke left side wala object this banta hai.

4. Default binding

```
1 fn();
```

- strict mode: this === undefined
- non-strict mode: generally this === globalThis

Fast debugging checklist

Kisi regular function call mein this nikalna ho, to ye order follow karo:

```
1 Kya `new` laga hai?
2     Haan → new object this hai
3
4 Nahi, kya call/apply use hua?
5     Haan → first argument this hai
6
7 Nahi, kya function object.property ke form mein call hua?
8     Haan → dot ke left wala object this hai
9
10 Nahi → default binding
```

15. Callback mein this kyun lose hota hai?

Suppose:

```
1  const infoForm = {
2      clickHandler(event) {
3          console.log(this);
4      },
5
6      init() {
7          button.addEventListener(
8              "click",
9              this.clickHandler
10             );
11        }
12    };
```

Ye line:

```
1  this.clickHandler
```

function ko invoke nahi kar rahi.

Invocation hoti to parentheses hote:

```
1  this.clickHandler()
```

Actual line sirf function reference callback system ko de rahi hai:

```
1  button.addEventListener("click", functionReference);
```

Baad mein callback ko event system invoke karega. Aapke visible code mein actual callback call-site nahi dikh raha.

Conceptually internal invocation kuch aisi ho sakti hai:

```
1 eventCallback(eventObject);
```

ya:

```
1 eventCallback.call(someObject, eventObject);
```

Isliye sirf ye dekhkar:

```
1 this.clickHandler
```

ye assume nahi karna chahiye ki callback ke andar bhi wahi outer this preserved rahega.

Function reference pass karna aur method call karna do alag operations hain.

16. Callback context fix karne ka old lexical variable pattern

Ek traditional solution:

```
1 const context = this;
2
3 button.addEventListener(
4   "click",
5   function handler(event) {
6     return context.clickHandler(event);
7   }
8 );
```

Jab ye line chalti hai:

```
1 const context = this;
```

current this value normal lexical variable context mein capture ho jati hai.

Callback ke andar:

```
1 context.clickHandler(event);
```

Call-site par context. present hai, isliye implicit binding ke through:

```
1 this === context
```

clickHandler() ko correct object milta hai.

Wrapper function ka this irrelevant kyun hai?

```
1 function handler(event) {  
2     return context.clickHandler(event);  
3 }
```

handler() ke andar this use hi nahi hua.

Woh this-aware nahi hai.

Event system handler ko kisi bhi this ke saath call kare, wrapper par farq nahi padega. Actual important call-site humne apne control mein bana liya:

```
1 context.clickHandler(event)
```

17. Arrow function ka main purpose: lexical this

Arrow function:

```
1 event => this.clickHandler(event)
```

Regular function ki tarah apna dynamic this create nahi karta.

Arrow function ke andar this nearest enclosing scope/function se liya jata hai.

Example:

```
1 function outer() {  
2     const inner = () => {  
3         console.log(this.value);  
4     };  
5  
6     return inner;  
7 }
```

Objects:

```
1 const one = {  
2     value: 42  
3 };  
4  
5 const two = {  
6     value: "sad face"  
7 };
```

Invocation:

```
1 const innerFn = outer.call(one);
```

outer() regular function hai. Explicit binding ke through:

```
1 outer ka this === one
```

inner arrow function isi surrounding this ko lexically use karega.

Ab:

```
1 innerFn.call(two);
```

Aap expect kar sakte ho ki call(two) ki wajah se this === two ho jayega.

Lekin nahi.

Arrow function apna this accept/assign nahi karta. .call(two) arrow ke lexical this ko change nahi kar sakta.

Result:

```
1 42
```

Arrow function ka this ab bhi one se aa raha hai.

18. Arrow function mein this lookup kaise hota hai?

Consider:

```
1 function outer() {  
2     const inner = () => {  
3         console.log(this.value);  
4     };  
5  
6     return inner;  
7 }
```

Jab arrow ke andar ye expression milta hai:

```
1 this.value
```

Arrow function ke paas apna this nahi hota.

Isliye lookup surrounding function tak jata hai:

```
1 outer()
```

outer regular function hai, isliye us invocation ka assigned this use hoga.

```
1 outer.call(one)
```

ki wajah se arrow ka lexical this one ko resolve karega.

Important correction to common wording

Log aksar bolte hain:

Arrow function apne parent ka this capture karta hai.

Ye beginner-level explanation useful hai, lekin “parent object” nahi. Arrow function lexically surrounding context ka this use karta hai.

Agar surrounding level bhi arrow hai, lookup aur upar jayega jab tak aisa enclosing context na mile jahan actual this available ho.

19. Event callback mein arrow solution

Old pattern:

```
1  const context = this;
2
3  button.addEventListener(
4    "click",
5    function (event) {
6      return context.clickHandler(event);
7    }
8  );
```

Arrow ke saath:

```
1  button.addEventListener(
2    "click",
3    event => this.clickHandler(event)
4  );
```

Arrow callback ka apna dynamic this nahi hai. Woh enclosing init() invocation ka this use karega.

Example:

```
1  const infoForm = {
2    init() {
3      button.addEventListener(
4        "click",
5        event => this.clickHandler(event)
6      );
7    },
8  }
```

```
9     clickHandler(event) {
10         console.log("Handled by:", this);
11     }
12 };
13
14 infoForm.init();
```

init() ko call kiya:

```
1 infoForm.init();
```

Isliye init ka this === infoForm.

Uske andar defined arrow wahi this use karega.

Baad mein event system arrow ko kis form mein call karta hai, arrow ka this usse change nahi hoga.

20. Arrow functions sirf short syntax nahi hain

Ye common misunderstanding hai:

Arrow functions normal functions ka shorter version hain.

Syntax shorter zaroor hai, lekin semantic difference important hai.

Regular function:

```
1 function handler() {
2     console.log(this);
3 }
```

Iska this call-site ke according dynamically assign hoga.

Arrow function:

```
1 const handler = () => {
2     console.log(this);
3 };
```

Iska apna this nahi hai. Ye surrounding lexical this use karega.

Chapter ki recommendation:

- lexical this chahiye, arrow function useful hai
 - dynamic this chahiye, regular function use karo
 - sirf kam characters type karne ke liye arrow choose mat karo
-

21. Nested arrows mein correct call-site kaise find karein?

Chapter ka contrived example conceptually ye sikhata hai:

```
1  globalThis.value = {
2      result: "Sad face"
3  };
4
5  function one() {
6      function two() {
7          const three = {
8              value: {
9                  result: "Hmmm"
10             },
11
12             fn: () => {
13                 const four = () => this.value;
14
15                 return four.call({
16                     value: {
17                         result: "OK"
18                     }
19                 });
20             },
21         };
22
23         return three.fn();
24     }
25
26     return two();
27 }
28
29 new one();
```

Question hai four() ke andar:

```
1  this.value
```

kahan se aayega?

Step 1: four

```
1  const four = () => this.value;
```

four arrow hai, apna this nahi.

```
1 four.call({ value: "OK" })
```

ka .call() arrow ka this set nahi kar sakta.

Isliye "OK" wala object irrelevant hai.

Step 2: fn

four ke surrounding mein fn hai:

```
1 fn: () => { ... }
```

fn bhi arrow hai. Iska bhi apna this nahi.

```
1 three.fn()
```

normally implicit binding lagata, lekin arrow fn dynamic this accept nahi karta. Isliye three ka "Hmmm" bhi use nahi hoga.

Step 3: two

Next enclosing regular function:

```
1 function two() {}
```

Ye call hua:

```
1 two()
```

Koi object prefix, explicit binding ya new nahi hai. Isliye default binding lagegi.

Chapter example non-strict context assume karta hai, jahan:

```
1 this === globalThis
```

Isliye:

```
1 this.value === globalThis.value
```

Result:

```
1 { result: "Sad face" }
```

Main lesson

Arrow function ke this ke liye arrow ka immediate call-site mat analyze karo. Enclosing chain mein upar jao aur nearest relevant regular-function context ko analyze karo.

22. bind() kya karta hai?

call() aur apply() function ko immediately invoke karte hain:

```
1 fn.call(obj);
2 fn.apply(obj, []);
```

bind() immediately invoke nahi karta.

```
1 const boundFn = fn.bind(obj);
```

Ye ek **naya function** return karta hai jiska this preset hai.

Example:

```
1 function showName() {
2     console.log(this.name);
3 }
4
5 const user = {
6     name: "Ruth"
7 };
8
9 const boundShowName = showName.bind(user);
10
11 boundShowName();
```

Output:

```
1 Ruth
```

Chahe plain invocation ho:

```
1 boundShowName()
```

original function ko user context ke saath run kiya jayega.

Event example:

```
1 button.addEventListener(
2     "click",
3     this.clickHandler.bind(this)
4 );
```

bind(this) current context ke saath ek naya bound function banata hai.

23. call, apply, aur bind mein difference

call

```
1 fn.call(obj, arg1, arg2);
```

- this explicitly set karta hai
- function immediately invoke karta hai

- arguments separately leta hai

apply

```
1 fn.apply(obj, [arg1, arg2]);
```

- this explicitly set karta hai
- function immediately invoke karta hai
- arguments array mein leta hai

bind

```
1 const newFn = fn.bind(obj, arg1);
```

- function immediately invoke nahi karta
 - naya wrapped/bound function return karta hai
 - future invocation ke liye this preset karta hai
-

24. Arrow function aur bind(this) same nahi hain

Common claim:

```
1 Arrow function = bind(this) ka short syntax
```

Chapter ke according ye accurate nahi hai.

Bound regular function

```
1 function showValue() {  
2     console.log(this.value);  
3 }  
4  
5 const obj = {  
6     value: 42  
7 };  
8  
9 const bound = showValue.bind(obj);  
10 bound();
```

Output:

```
1 42
```

Arrow wrapper

```
1 const fakeBound = (...args) =>  
2     showValue.apply(obj, args);
```

Normal invocation mein dono similar lag sakte hain:

```
1 bound();
2 fakeBound();
```

Dono 42 print kar sakte hain.

Lekin underlying semantics same nahi hain.

25. new bound function ko override kar sakta hai

Chapter ka nuanced case:

```
1 function thisAwareFn() {
2     console.log(`Value: ${this.value}`);
3 }
4
5 const obj = {
6     value: 42
7 };
8
9 const f = thisAwareFn.bind(obj);
```

Normal call:

```
1 f(); // Value: 42
```

Lekin:

```
1 new f();
```

new higher precedence rakhta hai. Is invocation ke liye newly created object this banega, obj nahi.

Naye empty object ke paas value property nahi hai, isliye:

```
1 Value: undefined
```

Ye precedence prove karta hai:

```
1 new binding > bound/explicit binding
```

26. Arrow function ko new ke saath use nahi kar sakte

```
1 const arrow = () => {};
2 new arrow();
```

Ye error dega.

Arrow function:

- apna this create nahi karta
- constructable function nahi hota
- .prototype based construction ke liye suitable nahi hota

Isliye:

```
1 new arrow()
```

allowed nahi hai.

Yahi ek clear difference hai:

```
1 const f = regularFunction.bind(obj);  
2 new f(); // possible
```

versus:

```
1 const g = () => {};  
2 new g(); // TypeError
```

Isliye arrow ko “hard-bound regular function” kehna conceptually wrong hai.

27. bind() ya arrow callback ek naya function banata hai

Ye do expressions different function references produce karte hain:

```
1 this.clickHandler.bind(this)  
1 event => this.clickHandler(event)
```

Har evaluation par generally naya function object milta hai.

Example:

```
1 const first = obj.method.bind(obj);  
2 const second = obj.method.bind(obj);  
3  
4 console.log(first === second); // false
```

Dono same original method par based hain, same object se bound hain, lekin references alag hain.

28. Event listener remove karne ki problem

Suppose:

```
1 button.addEventListener(  
2     "click",  
3     this.clickHandler.bind(this)  
4 );
```

Baad mein:

```
1 button.removeEventListener(  
2     "click",  
3     this.clickHandler.bind(this)  
4 );
```

Ye listener remove nahi karega, kyunki second bind() ne ek naya function create kar diya.

Conceptually:

```
1 const function1 = this.clickHandler.bind(this);  
2 const function2 = this.clickHandler.bind(this);  
3  
4 function1 !== function2;
```

Correct pattern:

```
1 this.boundClickHandler =  
2     this.clickHandler.bind(this);  
3  
4 button.addEventListener(  
5     "click",  
6     this.boundClickHandler  
7 );
```

Baad mein same reference:

```
1 button.removeEventListener(  
2     "click",  
3     this.boundClickHandler  
4 );
```

Main issue memory ka size nahi, balki **function identity** aur lifecycle management bhi hai.

29. Class fields mein arrow methods

Pattern:

```
1 class Point2d {  
2     x = null;  
3     y = null;
```

```

4
5     getDoubleX = () => this.x * 2;
6
7     constructor(x, y) {
8         this.x = x;
9         this.y = y;
10    }
11 }

```

Is pattern ka benefit:

```

1  const point = new Point2d(3, 4);
2
3  const getX = point.getDoubleX;
4  getX(); // 6

```

Method reference separate variable mein pass karne ke baad bhi expected instance context milta hai.

Lekin chapter is pattern ko critically dekhta hai.

Prototype method

Normal method:

```

1  class Point2d {
2      getDoubleX() {
3          return this.x * 2;
4      }
5  }

```

Ye method generally prototype par shared hota hai:

```

1  Point2d.prototype.getDoubleX

```

Multiple instances same function definition share kar sakte hain.

Arrow class field

```

1  getDoubleX = () => this.x * 2;

```

Ye per-instance function property create karta hai.

```

1  const p1 = new Point2d(3, 4);
2  const p2 = new Point2d(5, 6);
3
4  p1.getDoubleX === p2.getDoubleX; // false

```

Har instance ko apna arrow function milta hai.

Isse method detached hone par context predictable hota hai, lekin prototype sharing ka benefit kam ho jata hai.

30. Constructor mein methods bind karna

Another common pattern:

```
1  class Point2d {
2      constructor(x, y) {
3          this.x = x;
4          this.y = y;
5
6          this.getDoubleX =
7              this.getDoubleX.bind(this);
8      }
9
10     getDoubleX() {
11         return this.x * 2;
12     }
13 }
```

Prototype par original method hai:

```
1  Point2d.prototype.getDoubleX
```

Lekin constructor instance par same-name bound property create kar deta hai:

```
1  this.getDoubleX = boundFunction;
```

Instance property prototype method ko shadow kar deti hai.

Detached call work karega:

```
1  const point = new Point2d(3, 4);
2  const fn = point.getDoubleX;
3
4  fn(); // 6
```

Lekin har instance ke liye naya bound function create hoga.

Chapter author ko ye approach aur bhi less appealing lagti hai because:

1. prototype method define kiya
2. phir har instance par bound wrapper create kiya
3. dynamic this flexibility remove kar di
4. extra function identity manage karni padi

Ye author ka design opinion hai, universal ban nahi.

31. this ko fixed rakhna hai to closure consider karo

Chapter alternative factory function suggest karta hai:

```
1  function Point2d(px, py) {
2      const x = px;
3      const y = py;
4
5      return {
6          getDoubleX() {
7              return x * 2;
8          },
9
10         toString() {
11             return `(${x},${y})`;
12         }
13     };
14 }
```

Use:

```
1  const point = Point2d(3, 4);
2  const anotherPoint = Point2d(5, 6);
3
4  const f = point.getDoubleX;
5  const g = anotherPoint.toString;
6
7  f(); // 6
8  g(); // (5,6)
```

Yahan methods this use nahi kar rahe.

Woh lexical variables close over kar rahe hain:

```
1  x
2  y
```

Function objects kahin bhi pass karo, call-site badalne se lexical variables ka connection nahi badlega.

Closure ka benefit

Agar requirement hai:

Function hamesha isi fixed data/context ke saath kaam kare.

To closure naturally suitable ho sakta hai.

Dynamic this ka benefit

Agar requirement hai:

Same function ko different objects/contexts ke against reuse karna hai.

To dynamic this useful ho sakta hai.

Design decision

- 1 Dynamic reusable context chahiye?
- 2 Regular function + this consider karo
- 3
- 4 Fixed predictable context chahiye?
- 5 Lexical variables + closure consider karo

Chapter ka broader opinion hai ki agar har method ko arrow ya bind(this) se permanently fix karna pad raha hai, to ho sakta hai lexical closure-based design zyada direct ho.

32. Indirect function calls

Consider:

```
1  const init = point.init;  
2  init(3, 4);
```

Function detach ho gaya, isliye default binding.

Ek strange indirect call:

```
1  (1, point.init)(3, 4);
```

Comma operator expressions ko left-to-right evaluate karta hai aur final expression ka result return karta hai.

```
1  (1, point.init)
```

ka result function reference hai:

```
1  point.init
```

Lekin object reference ke through direct method call preserve nahi hua. Function value indirectly invoke hui.

Isliye ye effectively plain function call jaisa behave karta hai aur default binding apply ho sakti hai.

Another indirect form

```
1  (() => point.init)()(3, 4);
```

Breakdown:

```
1  () => point.init
```

ek function return karega jo point.init reference deta hai.

First call:

```
1  (() => point.init)()
```

result mein init function reference mila.

Second call:

```
1  result(3, 4)
```

plain function call hai. Isliye original point context lost hai.

33. Parentheses kab context nahi todte?

```
1  (point.init)(3, 4);
```

Aap soch sakte ho parentheses ki wajah se function indirect ho gaya.

Lekin JavaScript grammar is form ko effectively direct property call ki tarah treat karti hai:

```
1  point.init(3, 4);
```

Isliye this === point preserve hota hai.

Compare:

```
1  (point.init)(3, 4);    // point context retained
2  (1, point.init)(3, 4); // indirect, context lost
```

Parentheses alone aur comma expression same nahi hain.

34. IIFE mein this

IIFE:

```
1  (function () {
2      console.log(this);
3  })();
```

Function expression immediately plain call ke form mein invoke hui:

```
1 (functionValue)()
```

Isliye default binding apply hoti hai.

- strict mode mein undefined
- non-strict mode mein global context behavior mil sakta hai

Arrow IIFE different hoga:

```
1 (() => {  
2     console.log(this);  
3 })();
```

Arrow ka apna this nahi hai, isliye outer lexical this use hoga.

35. globalThis access ke historical tricks

Chapter purane patterns discuss karta hai jo standardized globalThis identifier available hone se pehle global object retrieve karne ke liye use hote the.

Function constructor trick

```
1 const gt = new Function("return this")();
```

Idea ye hai ki dynamically created function global context mein execute hota tha aur this ke through global object mil jata tha.

Indirect eval

```
1 function getGlobalThis() {  
2     return (1, eval)("this");  
3 }
```

eval() ko indirect form mein invoke kiya gaya:

```
1 (1, eval)
```

Phir string evaluate karke global this retrieve karne ki technique dikhayi gayi.

Problem

Dynamic code evaluation:

```
1 new Function(...)  
2 eval(...)
```

security policies, specifically strict Content Security Policy restrictions, ke under block ho sakti hai.

Chapter ye techniques modern recommendation ke roop mein nahi, balki this aur indirect invocation behavior explain karne ke liye dikha raha hai.

Modern code mein actual identifier available ho to direct use simple hai:

```
1 globalThis
```

36. Getter-based globalThis trick

Chapter ek unusual polyfill approach show karta hai:

```
1 function getGlobalThis() {
2   Object.defineProperty(
3     Object.prototype,
4     "__get_globalthis__",
5     {
6       get() {
7         return this;
8       },
9       configurable: true
10    }
11  );
12
13  const gt = __get_globalthis__;
14
15  delete Object.prototype.__get_globalthis__;
16
17  return gt;
18 }
```

Conceptually:

1. Object.prototype par temporary getter add hota hai
2. global lookup se property access hoti hai
3. getter ka this global object resolve karta hai
4. getter delete kar diya jata hai
5. global object return hota hai

Ye intentionally “gnarly” example hai. Daily application code mein aapko aisa karna nahi hai.

Is section ka main purpose hai dikhana ki accessor invocation mein bhi this context involved ho sakta hai.

37. Tagged template functions mein this

Normal tagged template:

```
1 function tagFn(strings) {  
2     console.log(strings);  
3 }  
4  
5 tagFn`Hello`;
```

Dikhne mein parentheses nahi hain, lekin tagFn invoke hota hai.

this behavior normal function calls se correspond karta hai.

Plain tagged function

```
1 tagFn`Hello`;
```

Ye context ke perspective se roughly:

```
1 tagFn(...);
```

jaisa behave karega.

Isliye default binding.

Object method as tag

```
1 const obj = {  
2     tagFn(strings) {  
3         console.log(this);  
4     }  
5 };  
6  
7 obj.tagFn`Hello`;
```

Ye context ke perspective se:

```
1 obj.tagFn(...);
```

jaisa behave karega.

Isliye implicit binding:

```
1 this === obj
```

Tagged templates mein this-aware tag functions rare ho sakte hain, lekin rule same call-site thinking follow karta hai.

38. Chapter ke kuch code typos aur confusing points

Provided chapter work in progress hai, aur snippets mein kuch likely typos hain.

Typo 1

```
1 this.theSubmitBtn =  
2   theFormElem.querySelector("button[type=submit]");
```

Yahan likely intended code:

```
1 this.theSubmitBtn =  
2   this.theFormElem.querySelector(  
3     "button[type=submit]"  
4   );
```

Original snippet mein theFormElem lexical variable declared nahi dikh raha.

Typo 2

Kuch places par:

```
1 this.submitBtnaddEventListener(...)
```

Likely intended:

```
1 this.submitBtn.addEventListener(...)
```

Property-name inconsistency

Chapter kabhi use karta hai:

```
1 theSubmitBtn
```

aur kabhi:

```
1 submitBtn
```

Concept samajhne ke liye dono ko same intended button-property maana ja sakta hai.

HTML escaping

Provided content mein:

```
1 =>  
2 <
```

dikh raha hai.

Actual JavaScript mein ye hoga:

```
1 =>  
2 <
```

Example:

```
1 const fn = () => true;
```

39. Practical decision guide

Normal object method call hai

```
1 obj.method();
```

Regular method appropriate hai:

```
1 const obj = {  
2     method() {  
3         console.log(this);  
4     }  
5 };
```

Same function different objects par reuse karna hai

Dynamic this useful hai:

```
1 function showName() {  
2     console.log(this.name);  
3 }  
4  
5 showName.call(user1);  
6 showName.call(user2);
```

Callback mein current outer this preserve karna hai

Arrow wrapper useful hai:

```
1 button.addEventListener(  
2     "click",  
3     event => this.handleClick(event)  
4 );
```

Stable callback reference bhi chahiye

Reference ko store karo:

```
1 this.handleClick =  
2     event => this.handleClick(event);  
3  
4 button.addEventListener(  
5     "click",
```

```
6     this.handleClick
7   );
```

Removal ke time:

```
1 button.removeEventListener(
2     "click",
3     this.handleClick
4 );
```

Fixed private state chahiye aur dynamic this nahi chahiye

Closure consider karo:

```
1 function createCounter() {
2     let count = 0;
3
4     return {
5         increment() {
6             count++;
7         },
8
9         getCount() {
10             return count;
11         }
12     };
13 }
```

Detached method bhi lexical state access kar sakta hai:

```
1 const counter = createCounter();
2 const increment = counter.increment;
3
4 increment();
5 increment();
6
7 counter.getCount(); // 2
```

40. Ek complete comparison

Regular function

```
1 function show() {
2     console.log(this.value);
```

```
3 }
```

this call-site se decide hoga:

```
1  const one = {
2      value: 1,
3      show
4  };
5
6  const two = {
7      value: 2,
8      show
9  };
10
11 one.show();          // 1
12 two.show();          // 2
13 show.call(one);      // 1
14 show.call(two);      // 2
```

Arrow function

```
1  function createShow() {
2      return () => {
3          console.log(this.value);
4      };
5  }
```

Outer invocation this decide karega:

```
1  const one = {
2      value: 1
3  };
4
5  const two = {
6      value: 2
7  };
8
9  const show = createShow.call(one);
10
11 show();              // 1
12 show.call(two);      // still 1
```

Bound function

```
1  const boundShow = showRegular.bind(one);
```

Normal calls mein preset context:

```
1 boundShow();           // one
2 boundShow.call(two);    // still one
```

Lekin constructable bound function ke certain cases mein new higher precedence rakh sakta hai.

41. Chapter ka real message

Chapter sirf ye nahi sikhata ki this ke four rules kya hain. Iska deeper design message hai:

1. this ko intentionally use karo

Har object method mein automatically this use karna zaroori nahi.

2. Dynamic behavior ki actual need identify karo

Agar same function ko different context ke saath reuse karna hai, this powerful hai.

3. Fixed context ho to lexical scope simpler ho sakta hai

Agar function ko hamesha same variables/data ke saath kaam karna hai, closures direct solution ho sakte hain.

4. Callback pass karte waqt method association preserve nahi hoti

```
1 someUtility(obj.method);
```

sirf function reference pass karta hai. obj automatically function ke saath permanently attached nahi hota.

5. Arrow ka main semantic feature lexical this hai

Arrow ko only short function syntax mat samjho.

6. bind() free operation nahi hai

Ye naya function reference create karta hai, jisko store aur manage karna pad sakta hai.

7. Function definition nahi, invocation analyze karo

Regular functions ke liye this answer karne ka fundamental method call-site analysis hai.

Final cheat sheet

```
1 // 1. Implicit binding
```

2	<code>obj.fn();</code>
3	<code>// this = obj</code>
1	<code>// 2. Explicit binding</code>
2	<code>fn.call(obj, a, b);</code>
3	<code>fn.apply(obj, [a, b]);</code>
4	<code>// this = obj</code>
1	<code>// 3. New binding</code>
2	<code>new Fn();</code>
3	<code>// this = newly created object</code>
1	<code>// 4. Default binding</code>
2	<code>fn();</code>
3	<code>// strict mode: this = undefined</code>
4	<code>// non-strict behavior: this = globalThis</code>
1	<code>// 5. Arrow function</code>
2	<code>const fn = () => this.value;</code>
3	<code>// no own dynamic this</code>
4	<code>// surrounding lexical this is used</code>
1	<code>// 6. Bound function</code>
2	<code>const boundFn = fn.bind(obj);</code>
3	<code>// new function returned</code>
4	<code>// normal calls use obj as this</code>

Ek line mein pura chapter

Regular function ka this us function ke call-site se decide hota hai, jabki arrow function ka apna this nahi hota aur woh surrounding lexical this use karta hai.

Explanation approach

Maine chapter ko sirf definitions ke form mein nahi, balki call-site analysis ke order mein organize kiya hai. Sabse important improvement ye hai ki this ko “function kis object mein likha hai” se associate karne ke bajay “function ko exactly kaise invoke kiya gaya” se analyze kiya jaye.